CS H Practice Set Answers

Writer: Hans 10-1

Editor: Jingzhe 10-8

Range: Network, Database, Machine Learning, Recursion

Note: (Review Guide/Review Practices/Practice Set) are UNOFFICIAL sources of reference and do NOT include or hint at questions in the actual exam!

1 Concepts

1. B) To classify data based on the majority vote of its K nearest neighbors
2. C) Through cross-validation and balancing noise sensitivity with pattern capture
3. B) An attribute or variable describing the input data
4. C) Support Vector Machine (SVM)
5. B) To train the model by providing input-output pairs
6. B) To measure the similarity between data points
7. B) It will cause a stack overflow
8. B) Stack
9. B) To store, retrieve, and manage structured data efficiently
10. B) A field that links to the primary key of another table
11. D) 'SELECT'
12. B) It improves IP address utilization and network management
13. B) To display the route packets take to a destination
14. A) "10.0.0.0 – 10.255.255.255"
15. A) To process and interpret visual data like images and videos

2 Coding

Problem 1: Factorial Function

(a) The base case for the factorial function is $0! = 1$. In fact, $1!$ is also 1 (since $1 \times 0! = 1$). We need a base case to stop the recursion; without it, the function would call itself forever (or until a stack overflow). Defining $0! = 1$ provides a stopping point for the recursive definition and is consistent with the mathematical definition of factorial.

(b) The completed recursive implementation in Python is:

```
def factorial(n):
    if n == 0:          # base case
        return 1
    else:
        return n * factorial(n-1)
```

Here, if n is 0, the function returns 1. Otherwise, it returns n multiplied by the factorial of $n-1$, directly mirroring the recursive definition $n! = n \times (n-1)!$.

(c) The recursive calls for `factorial(5)` unfold as follows:

$$\begin{aligned}
factorial(5) &= 5 \times factorial(4) \\
&= 5 \times (4 \times factorial(3)) \\
&= 5 \times (4 \times (3 \times factorial(2))) \\
&= 5 \times (4 \times (3 \times (2 \times factorial(1)))) \\
&= 5 \times (4 \times (3 \times (2 \times (1 \times factorial(0))))) \\
&= 5 \times (4 \times (3 \times (2 \times (1 \times 1)))) \quad (\text{since } factorial(0) = 1) \\
&= 5 \times (4 \times (3 \times (2 \times 1))) \\
&= 5 \times (4 \times (3 \times 2)) \\
&= 5 \times (4 \times 6) \\
&= 5 \times 24 \\
&= 120 \ ,
\end{aligned}$$

so `factorial(5)` returns 120. The function calls itself down from 5 to 0, then the results propagate back up, multiplying together to give 120.

Problem 2: Fibonacci Sequence

(a) Using the recursive definition of Fibonacci:

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2) \text{ for } n \geq 2 \ ,$$

we can compute:

- $F(2) = F(1) + F(0) = 1 + 0 = 1$,
- $F(3) = F(2) + F(1) = 1 + 1 = 2$,
- $F(4) = F(3) + F(2) = 2 + 1 = 3$.

(For completeness, $F(5)$ would be $F(4) + F(3) = 3 + 2 = 5$.)

(b) A correct recursive implementation of the Fibonacci function in Python is:

```
def fib(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fib(n-1) + fib(n-2)
```

This function handles the base cases $n = 0$ and $n = 1$ by returning 0 and 1 respectively. For any $n \geq 2$, it recursively computes `fib(n-1)` and `fib(n-2)` and returns their sum, implementing the definition of $F(n)$.

(c) The naive recursive approach is elegant but inefficient because it recomputes values many times. For example, to compute `fib(5)`, the function will compute `fib(4)` and `fib(3)`. But `fib(4)` itself calls `fib(3)` and `fib(2)`. That means `fib(3)` is computed twice (once directly as `fib(3)` and once as part of `fib(4)`), and `fib(2)` is also computed multiple times. In general, `fib(n)` ends up recalculating lower Fibonacci numbers repeatedly. This exponential blow-up makes it very slow for larger n. For instance, `fib(5)` will end up computing: - `fib(3)` twice (as noted above), - `fib(2)` three times (as part of the calls for `fib(3)` and `fib(4)`, etc.),

and so on. The overlapping subproblems lead to a lot of redundant computation, which is why pure recursion is inefficient for large Fibonacci numbers.

Problem 3: Permutations of a List

(a) The six permutations of `[1, 2, 3]` are:

`[1, 2, 3]`, `[1, 3, 2]`, `[2, 1, 3]`, `[2, 3, 1]`, `[3, 1, 2]`, `[3, 2, 1]`.

Each of these is a unique ordering of the list [1,2,3].

(b) One possible implementation of `insert_at_all_positions(x, perm)` is:

```
def insert_at_all_positions(x, perm):
    result = []
    for i in range(len(perm) + 1):
        # Insert x at position i
        new_perm = perm[:i] + [x] + perm[i:]
        result.append(new_perm)
    return result
```

This function returns a list of lists, each being the list `perm` with element `x` inserted at a different position (from index 0 up to the end of the list). For example, using this helper: `insert_at_all_positions(3, [1, 2])` returns `[[3, 1, 2], [1, 3, 2], [1, 2, 3]]`, which matches the example given.

(c) Using the above helper, we can implement `permute(lst)` recursively as follows:

```
def permute(lst):
    if len(lst) == 0:
        return [[]]   # Base case: one permutation of an
            empty list (the empty list)
    # Recursive case:
```

```
    first = lst[0]
    rest = lst[1:]
    perms_of_rest = permute(rest)          #
        Permutations of the sublist without the first
        element
    result = []
    for perm in perms_of_rest:
        # insert the first element into every possible
            position of each smaller permutation
        result.extend(insert_at_all_positions(first,
            perm))
    return result
```

How it works: When `permute` is called on a list, it removes the first element and recursively finds permutations of the remaining elements. Then, it uses `insert_at_all_positions` to insert the first element into every possible position in each of those smaller permutations. The base case returns `[[]]` (a list containing an empty list) as the only permutation of an empty list. This ensures that when the recursion bottoms out, it starts building back up correctly. The final result is a list of all permutations of the input list.

Problem 4: Power Set (All Subsets)

(a) The subsets of $\{1, 2\}$ are: the **empty set** (no elements), $\{1\}$, $\{2\}$, and $\{1, 2\}$. In list form (if we represent subsets as Python lists), we could write these as: `[]`, `[1]`, `[2]`, `[1, 2]`. (It's important to include the empty set and the full set in the list of all subsets.)

(b) The relationship between subsets of $\{2, 3, \ldots, n\}$ and subsets of $\{1, 2, \ldots, n\}$ is as follows:

Take all subsets of $\{2, 3, \ldots, n\}$ (those that do not include 1). For each of those subsets, you can form a new subset of $\{1, \ldots, n\}$ by adding the element 1 to it. This generates all the subsets that *do* include 1. Together, - the subsets without 1 (which are just the subsets of the smaller set), and - the subsets with 1 (which you get by adding 1 to each subset of the smaller set)

comprise all subsets of the larger set. For example, if $n = 3$: subsets of $\{2, 3\}$ are $\{\}, \{2\}, \{3\}, \{2, 3\}$. To get subsets of $\{1, 2, 3\}$, take all those and then add $\{1\}$ to each of them, yielding $\{1\}, \{1, 2\}, \{1, 3\}, \{1, 2, 3\}$, which combined with the original four gives eight subsets total.

(c) A recursive implementation of `subsets(lst)` is:

```
def subsets(lst):
    if len(lst) == 0:
        return [[]]   # Base case: the only subset of an
            empty list is the empty list
```

```
    first = lst[0]
    rest = lst[1:]
    # Recursively get subsets of the remaining elements
    subsets_without_first = subsets(rest)
    # Now create subsets that include the first element
    subsets_with_first = []
    for sub in subsets_without_first:
        subsets_with_first.append([first] + sub)
    # Combine the subsets without first and with first
    return subsets_without_first + subsets_with_first
```

Explanation: If `lst` is empty, the function returns `[[]]`. Otherwise, it splits the list into `first` and `rest`. It recursively finds all subsets of `rest`. Then for each such subset, it creates a new subset by adding `first` at the beginning. The result is the union of all subsets that don't contain `first` and all those that do contain `first`. This corresponds exactly to the include/exclude idea from part (b).

Problem 5: Binary Search (Divide and Conquer)

(a) Binary search works by repeatedly dividing the search interval in half. You compare the target value to the element at the middle of the current range:

- If the target equals the middle element, the search is successful. - If the target is smaller than the middle element, you "discard" the upper half of the array and continue searching in the lower half. - If the target is larger than the middle element, you discard the lower half and continue in the upper half.

By always choosing the half that could contain the target, the algorithm significantly reduces the search space at each step (divide-and-conquer).

(b) The completed recursive binary search implementation is:

```
def binary_search_rec(arr, target, low, high):
    if low > high:
        return -1  # target not found
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        # search in the right half
        return binary_search_rec(arr, target, mid+1,
            high)
    else:
        # search in the left half
        return binary_search_rec(arr, target, low, mid
            -1)
```

```
def binary_search(arr, target):
    return binary_search_rec(arr, target, 0, len(arr)-1)
```

Here, `binary_search` is a wrapper that calls the recursive helper with the initial full range of indices. The helper `binary_search_rec` checks the base case (`low > high` means the target is not in the array), computes the middle index, and then recursively searches the appropriate half of the array.

(c) **Trace for target = 7 in [1,3,5,7,9]:** The function will find 7 in two recursive calls:

- **Call 1:** `low = 0`, `high = 4`. Compute `mid = (0+4)//2 = 2`. The element at index 2 is 5. Since 7 (target) is greater than 5, the algorithm will search in the right half. Now it calls `binary_search_rec(arr, 7, 3, 4)`.
- **Call 2:** `low = 3`, `high = 4`. Compute `mid = (3+4)//2 = 3`. The element at index 3 is 7, which matches the target, so the function returns 3.

The initial `binary_search` call then returns 3 as well. Thus, the index of 7 in the list is $\boxed{3}$ (assuming 0-based indexing). The sequence of `(low, mid, high)` was (0,2,4) then (3,3,4).

Problem 6: Height of a Binary Tree

(a) In the example tree, the longest path from the root to a leaf goes through 3 nodes. For instance, one such longest path is $10 \to 5 \to 7$. (The other path $10 \to 15 \to 12$ also has 3 nodes.) Therefore, the height of the tree is 3. By definition, an empty tree has height 0 and a tree with just one node has height 1. Here the root-to-leaf paths each involve 3 nodes, so height = 3.

(b) One can compute the height of a binary tree recursively by looking at the heights of the subtrees. A correct implementation is:

```
def height(root):
    if root is None:
        return 0
    left_height = height(root.left)
    right_height = height(root.right)
    # height is 1 (for the root itself) plus the larger
        of the two subtree heights
    return 1 + max(left_height, right_height)
```

This function returns 0 if the node is `None` (base case, empty tree). Otherwise, it computes the height of the left subtree and the right subtree

recursively, and returns 1 plus the greater of the two. This corresponds to the formula:

$$\text{height(root)} = 1 + \max(\text{height(root.left)},\ \text{height(root.right)})\ ,$$

which captures the definition of tree height.

Problem 7: Counting Islands in a Grid (Recursive Flood Fill)

(a) In the example grid, the islands (groups of connected 1s) are located as follows:

- **Island 1:** The cluster at the top-left, comprising cells (0,0) and (1,0) (using 0-based indices for rows and columns). These two 1s are vertically connected. - **Island 2:** The single cell at (0,3) is a 1 isolated by water (0s) around it. - **Island 3:** The cluster involving cells (1,2) and (2,2). These two are vertically adjacent, forming an island.

There are no other 1s connected horizontally or vertically beyond these groups, so the total number of islands is 3.

(b) A recursive helper function to "flood-fill" or explore an island can be written as:

```
def exploreIsland(grid, r, c):
    # Check for out-of-bounds or if current cell is not
        land
    if r < 0 or r >= len(grid) or c < 0 or c >= len(grid
        [0]):
        return
    if grid[r][c] == 0:
        return
    # Mark the current land cell as visited (e.g., sink
        it to water)
    grid[r][c] = 0
    # Recursively explore all four neighboring cells
    exploreIsland(grid, r+1, c)  # down
    exploreIsland(grid, r-1, c)  # up
    exploreIsland(grid, r, c+1)  # right
    exploreIsland(grid, r, c-1)  # left
```

This function checks bounds and whether the current cell is land. If it is land (1), it marks it as visited by setting `grid[r][c]` to 0 (turning it into water to indicate it's been counted), then calls itself on all four neighbors. Diagonal neighbors are not considered in this problem, as per the typical definition of "adjacent" in grid island problems. After `exploreIsland` returns, all cells connected to the original cell at (r, c) will have been marked.

(c) Using the helper above, the `count_islands(grid)` function can be implemented as:

```
def count_islands(grid):
    if not grid:
        return 0
    num_islands = 0
    rows = len(grid)
    cols = len(grid[0])
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:       # found an
                unvisited land cell
                num_islands += 1      # increment island
                    count
                exploreIsland(grid, r, c)  # mark the
                    entire island as visited
    return num_islands
```

This function iterates through every cell in the grid. When a cell with value 1 is found, it means a new island has been discovered. It increments the counter and calls `exploreIsland` on that cell to visit all cells in that island (changing them to 0 to mark them). By the time the helper returns, that island is fully "sunk", so no cell in it will trigger another count. The loop continues until all cells are checked. The result `num_islands` is the total number of islands found.

For instance, on the example grid provided, the function would count 3 islands, as reasoned in part (a). After running, all the 1s would be turned to 0 (the grid would become all 0s) since `exploreIsland` marks visited land as 0.